

ITF20219-1 25V
Datanettverk

Andreas Isaksen

February 13, 2025

Teoridel

- 1) TCP er en protokoll for pålitelig overføring av data. Hva vil det si at den er pålitelig, og hvilke mekanismer benytter den for å oppnå dette?

Svar:

Det at en dataoverføringprotokoll kan defineres som pålitelig, bygger på at protokollen har et tilfredsstillende oppsett av underliggende prinsipielle løsninger som sikrer en pålitelig overføring.

En *TCP* protokoll har flere slike underliggende mekanismer som kvalifiserer den som en pålitelig dataoverføringprotokoll.

Et TCP datagram(pakke) er bygget opp med følgende mekanismer:

- **Sekvensnummerering:**

Hver pakke får et sekvensnummer, som sikrer skal sikre at oversendte datagram er;

- I riktig rekkefølge.
- Ikke duplikater.
- Uten feil.
- Uten tap.

- **ACK/Bekreftelse:**

Når det sendes en pakke, svarer mottaker med et *ACK* signal tilbake som bekrefter at pakken er mottatt uten feil.

- **Tidsavbrudd:**

Om en avsender ikke får tilbake en *ACK* innen en viss tid, vil den sende den tilhørende datapakken på nytt. Det finnes flere grunner til at en *ACK* ikke blir returnert, men avsender skiller ikke på disse og sender pakke på nytt uavhengig av grunn, gitt at *timeout* for pakkens *ACK* er utgått.

- **Flytkontroll(Flow control):**

Dette er en verdi som blir satt av mottaker ved starten av dataoverføringen, som en del av *'handshake'* steget. Her forteller mottaker hvor raskt den er villig til å motta pakker. Dette er for å unngå potensiell overbelastning og data tap hos mottaker.

- **Congestion control:**

TCP tilpasser sendefrekvensen basert på nettverksforhold for å unngå overbelastning på nettløpe.

Dette gjør den ved bruk av *AIMD*(Additive Increase, Multiplicative Decrease) algoritmen, som fungerer ved å øke hastigheten på oversending ved hver vellykkede oversending, og halvere hastigheten ved hver feilede pakke.

- 2) Hva menes med 'effektivitet' på en dataoverføring?

Svar:

Dette refererer til målingen av mengden med data som blir oversendt i forhold til den teoretisk høyeste kapasiteten til båndbredden. Det er flere faktorer som påvirker den antatte effektiviteten. Slik som,

- Definisjoner i TCP protokollens header.
- Forsinkelser på pakkeoversendinger over nettverket (*RTT*(Round Trip Time)).
- Størrelse på datagram.
- Tap av data, slik som manglende ACK og gjentatte sendinger av samme datagram.

- 3) Hva er effektiviteten på en forbindelse til Princeton University? Anta at du har en kontinuerlig overføring, med et sendevindu på 16 pakker. Pakkestørrelsen er på 1024 byte (1 byte = 8 bit), og datahastigheten ut av din PC er 50 Mbit/s. Forsinkelsen til Princeton University kan du lese ut av en ping til maskinen:

Pinging www.princeton.edu [140.180.223.42] with 32 bytes of data:

Reply from 140.180.223.42: bytes=32 time=123ms TTL=242

Reply from 140.180.223.42: bytes=32 time=124ms TTL=242

Reply from 140.180.223.42: bytes=32 time=123ms TTL=242

Reply from 140.180.223.42: bytes=32 time=124ms TTL=242

Ping statistics for 140.180.223.42:

Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),

Approximate round trip times in milli-seconds:

Minimum = 123ms, Maximum = 124ms, Average = 123ms

Svar:

Vi kan regne ut effektiviteten med følgende formel:

$$Effektivitet = \frac{Sendevindu(antall\ pakker)}{Datahastighet \cdot snitt\ RTT}$$

I oppgaven har vi fått følgende informasjon:

- Sendevindu = 16 pakker på 1024byte hver.
- Datahastighet = 50 Mbit/s.
- Snitt RTT = 123ms.

Med dette kan vi gjøre følgende utregninger.

$$Sendevindu = 16 \cdot (1024 \cdot 8) = 16 \cdot 8192 = 131'072bit$$

$$Datahastighet = 50 \cdot 10^6 = 50'000'000s$$

$$\text{snitt } RTT = 123ms = 0.123s$$

Med dette kan vi fylle ut formelen:

$$Effektivitet = \frac{131'072}{50'000'000 \cdot 0.123} \approx 0.0213 = 2.13\%$$

I dette tilfellet blir 2.13% av den potensielle båndbredden utnyttet. Dette kommer da av begrensinger i sendevindu og RTT.

- 4) Hva blir den effektive (opplevd) datahastigheten på forbindelsen til Princeton University?

Svar:

Basert på data for ovennevnte oppgave kan vi regne ut den effektive hastigheten på følgende måte.

$$Effektiv hastighet = \frac{\text{sendevindu}}{RTT}$$

$$Effektiv hastighet = \frac{131'072bit}{0.123s}$$

$$Effektiv hastighet \approx 1'065'626.016bit/s$$

$$\frac{1'065'626.016bit/s}{10^6} \approx 1.066Mbit/s$$

Den opplevde effektive hastigheten i dette tilfellet blir dermed omtrent 1.066Mbit/s

- 5) Hvordan virker flytkontrollen mellom to host på transportlaget? Anta at det brukes TCP protokollen.

Svar:

Flytkontrollen i TCP fungerer ved å sørge for at mottaker ikke blir overbelastet.

-

- 6) Mottaker deklarerer i et *receiver window*(rwnd) hvor høy hastighet den kan ta imot datagram.
- 7) Sender gjør en hastighetbegrensing basert på dette vinduet.
- 8) Avsender regulerer rwnd basert på om mottaker blir overbelastet i henhold til AIMD algoritmen. Med denne løsningen kan vi forhindre pakketap, samt sikre en stabil overføring mellom sender og mottaker.

Praktisk del

Oppgave 1

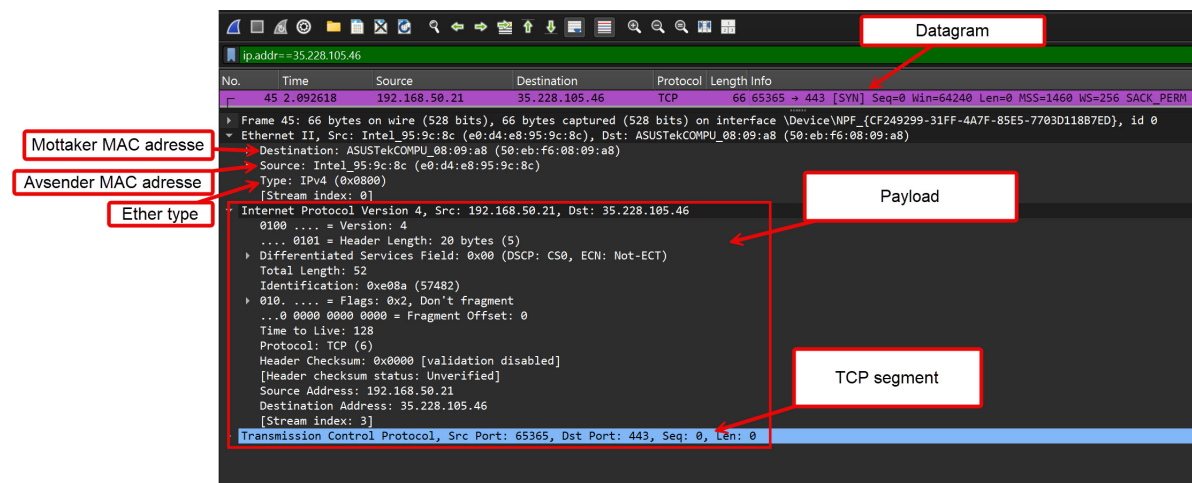


Figure 1: Finn.no TCP

I bildet over har jeg tegnet pekere til de forskjellige delene i en Ethernet header tilhørende en datapakke fra avsender til mottaker. Dette er den første pakken i en sekvens (Seq=0).

Jeg hadde problemer med å finne CRC biten av pakken, og etter litt leting på nettet fant jeg ut at CRC ofte blir filtrert bort i nettverk adaptore før pakken blir hentet opp av *Wireshark* programvaren. Derfor er ikke dette tatt med i fremstillingen.

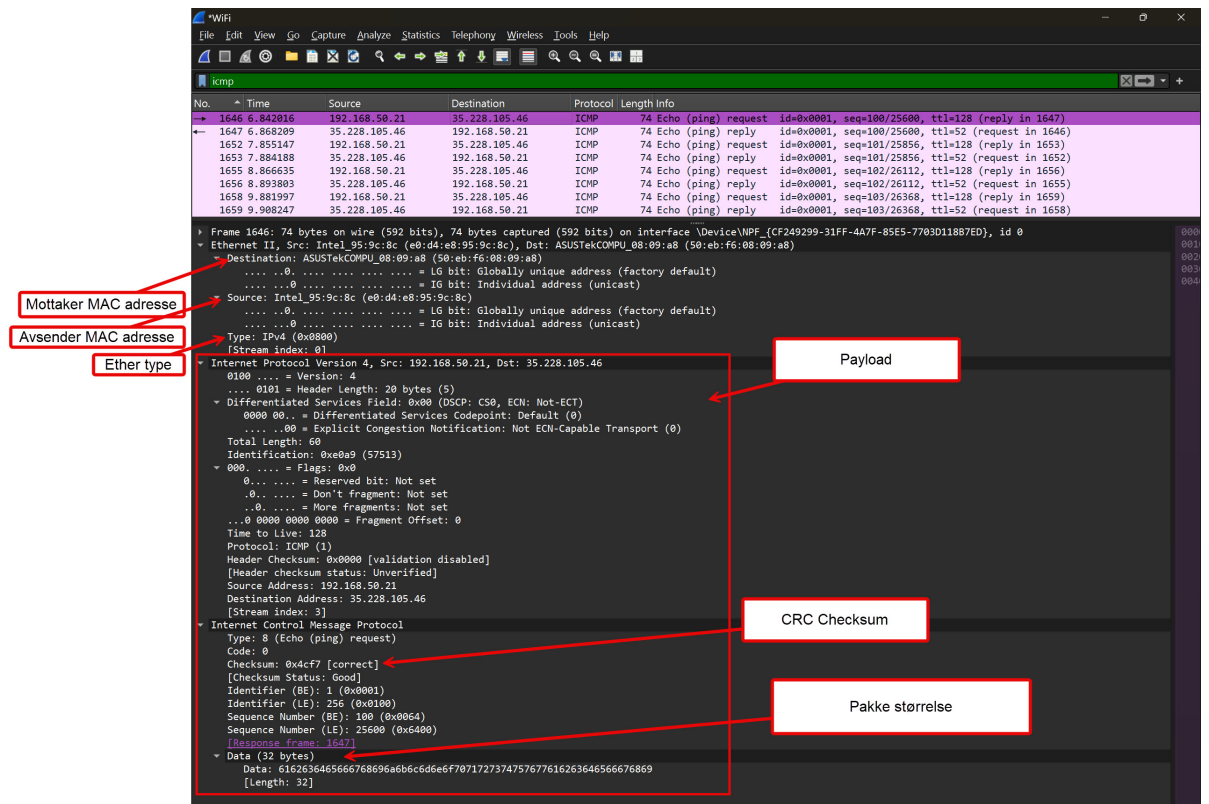


Figure 2: Finn.no ICMP

I bildet over har jeg tegnet pekere til de forskjellige delene i en Ethernet header tilhørende en datapakke på en ping forespørsel fra avsender til mottaker. Dette er den første pakken i en ping sekvens(seq=100). Med *Internet Control Message Protocol* fikk jeg også med *CRC Checksum* som er henvist med en peker i bildet.

Oppgave 2

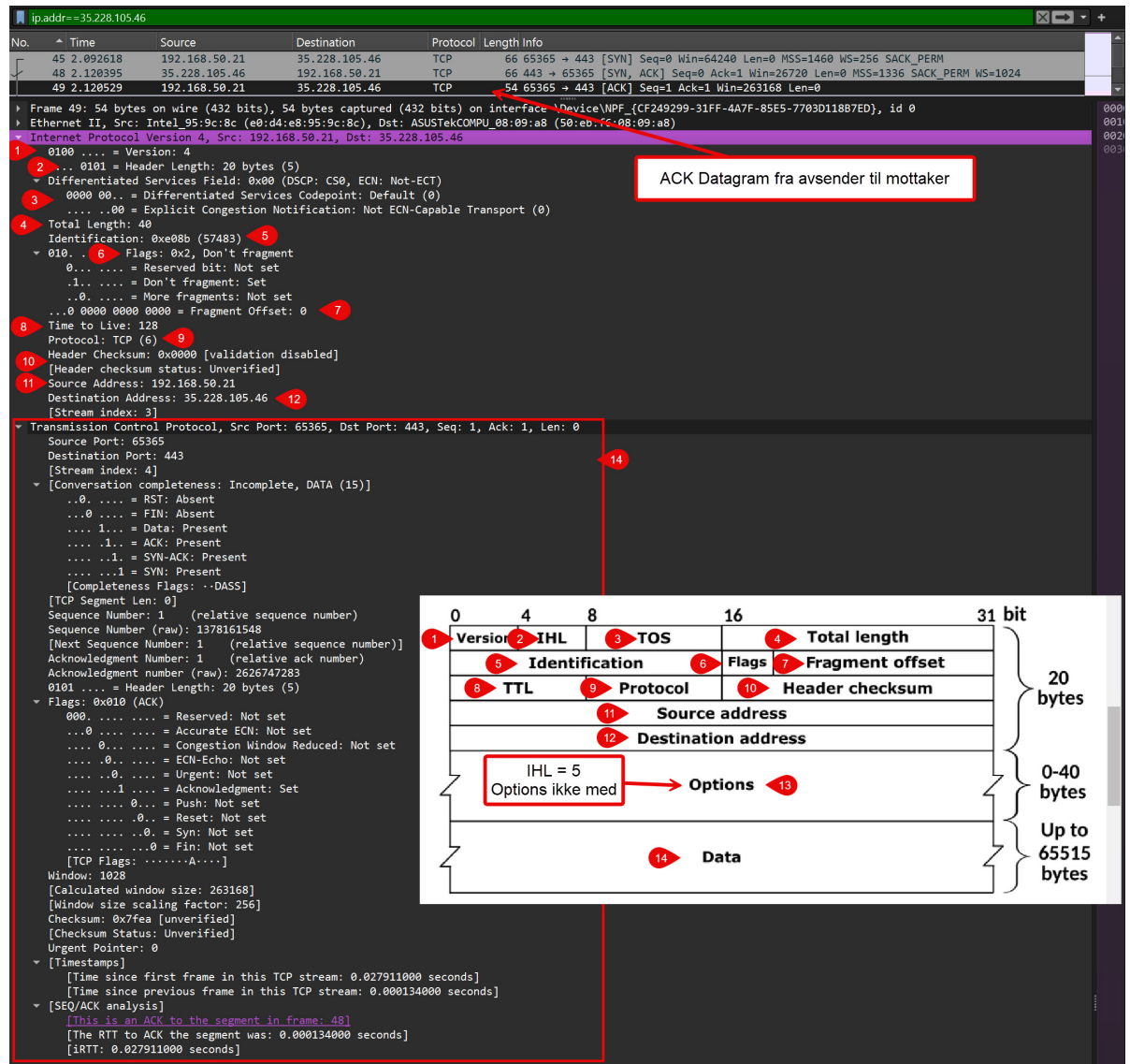


Figure 3: Finn.no ACK IPv4

I bilde over har jeg nummerert de forskjellige seksjonene i *IP Header* i forhold til bilde illustrasjonen.

1. Version – Spesifiserer IP versjonen. Består av 4 bits.

2. IHL – Indikerer lengden på en IPv4 *header*. Består av 4 bits. Minimum verdi er 5 (20 bytes) og maksimum er 15 (60 bytes).
3. TOS – Står for '*Type Of Service*' og spesifiserer prioritet på pakke. Består av 8 bits.
4. Total length – Spesifiserer lengden på hele pakken (*header* og *data*) i bytes. Består av 16 bits, som tillater pakker på opp til 65'353 bytes.
5. Identification – Identifikator på pakke for fragmentering og gjenoppbygning av pakke. Består av 16 bits.
6. Flags – Flagg kontroll for fragmentering. Består av 3 bits.
7. Fragment offset – Indikator for et fragments plassering. Brukes til gjenoppbygning av pakker. Består av 13 bits.
8. TTL – Står for *Time To Live* og er en løsning for å unngå at en pakke blir vandrende i en evig loop. TTL blir redusert med 1 for hvert ledd pakken er i gjennom. Om TTL==0 blir pakken stoppet og det sendes et ICMP signal tilbake til avsender om at pakken er stoppet. Består av 8 bits.
9. Protocol – Spesifiserer transport lag protokollen som brukes i data feltet. Består av 8 bits.
10. Header checksum – En løsning for feil detektering i IP header. Består av 16 bits.
11. Source address – Holder på IP adressen til avsender. Består av 32 bits.
12. Destination address – Holder på IP adressen til mottaker. Består av 32 bits.
13. Options – Dette feltet brukes for potensiell ekstra funksjonalitet ved behov. Her kan det spesifiseres ting som loggføring av rute, timestamp, sikkerhet og mye mer. Dette feltet er kun tilstede om IHL_i5. Består av varierende antall bits.
14. Data – Dette er pakkens faktiske *payload* som inneholder data tilhørende transport laget. Består av varierende antall bits.

Oppgave 3

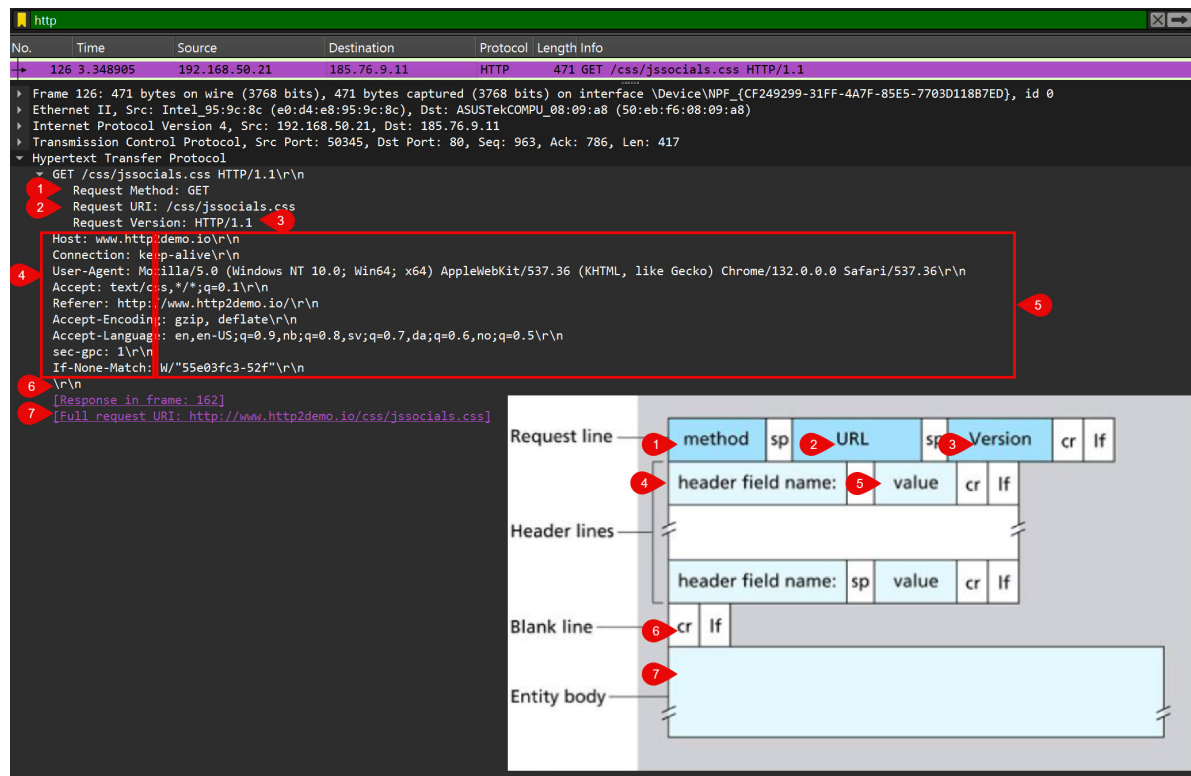


Figure 4: HTTP GET

I bildet over har vi et eksempel på en *Hypertext Transfer Protocol* i form av en GET metode.

Bilde er nummerert med pekere som korresponderer med illustrasjonen i fra oppgaven.

1. Method – Her blir det deklartert hvilken metode denne pakken bruker. I dette tilfellet er dette en GET metode som sender en forespørsel.
2. URL – Dette feltet spesifiserer URI feltet til innholdet som GET metoden etterspør.
3. Version – Her defineres hvilken HTTP versjon GET metoden bruker.
4. Header field name – Disse feltene spesifiserer metadata i pakken.
5. Value – Dette er verdiene som blir definert for hvert enkelt felt av meta-data.

6. Blank line – Et linjeskift som avslutter header på pakke.
7. Entity body – Dette feltet er tomt i en GET forespørsel, da GET etterspør data fremfor å sende den.

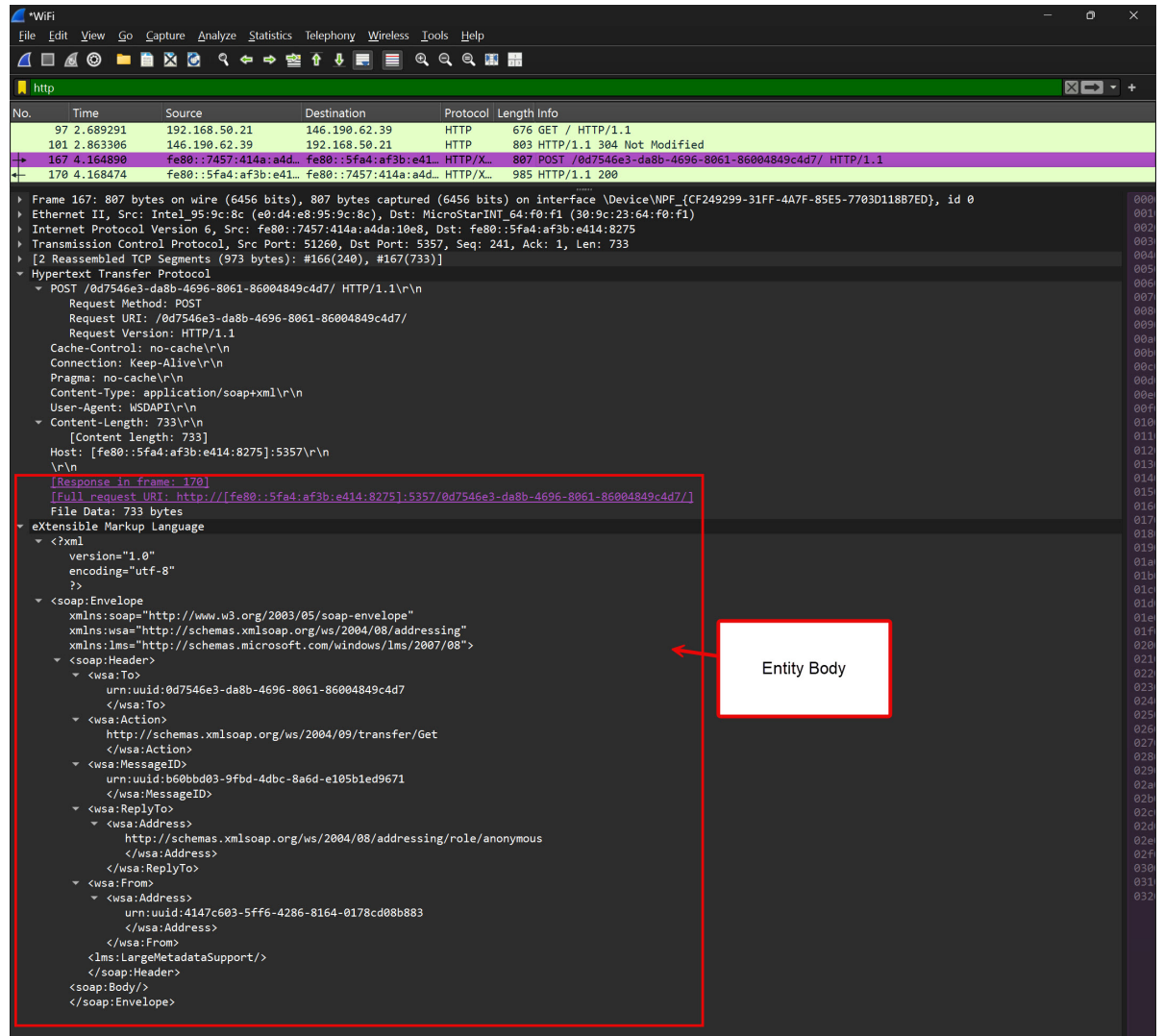


Figure 5: http POST

I bilde over her ser vi en respons pakke i form av POST. Vi kan se at strukturen på en GET og en POST pakke er relativt like. Bare noen endringer i forskjell på metodekall og metadata.

Den eneste store forskjellen er at POST ikke er tom i *Entity body*, men inneholder den etterspurte informasjonen til GET forespørselen (samt eventuelle videre referanser til mer data, slik som i denne pakken). Nevner at denne POST pakken ikke korresponderer med den illustrerte GET pakken over, da jeg ikke fikk korrekt data fra GET kallet og måtte hente denne POST pakken et annet sted.